ORDEROPS AI · AUTO HEALING

Moa

주문은 AI가 모으고,
정산서는 한 장으로

모아 · 이홍진 · 2026.11.09

TEAM

함께 만든 사람들

이성우

팀장 · PM
서비스 · 멀티 에이전트

- 주문 정리 서비스 본체 (업무 층 에이전트 · 툴 · API)
- 사용자 화면 · 운영 대시보드
- PM 문서

노지훈

Auto Healing

- 장애 감지 · 진단 · 복구 · 검증 워크플로
- 고장 주입기
- 사건 기록 · 지표

강민우

QA

- 테스트 작성
- 품질 측정 · 정답 데이터
- 배포 차단 · Issue 자동 할당 실증

이홍진

CI / CD

- 자동 테스트 파이프라인 (GitHub Actions)
- 컨테이너 구성 (공용 스택 + 앱 A/B · HAProxy 게이트웨이)
- AWS 클라우드 (EC2) 배포

백승주

측정 · 검증

- 장애 주입 · 측정 · 독립 검증
- 데모 총괄 · 제출 책임
- CI · 시연 영상 · 발표자료 보조

PAINPOINT

사내 티타임, 인원이 늘면 생기는 일

주 1회 티타임

분위기 좋은 카페에서 아이스브레이킹, 그다음 업무 이야기

3~4명이 딱 좋다

그 이상이면 카페에 들어가기도, 다 같이 앉기도 어렵다

결국 회의실로

커피는 배달, 주문은 팀장님이 공유한 구글 스프레드시트에

PAINPOINT

스프레드시트 주문, 문제없어 보이지만

이름	메뉴
A	아아 톨
B	뜨아 톨
C	아이스 바닐라라떼 톨
D	따뜻한 카페모카 그란데 휘핑크림 빼주세요
E	뜨아 톨
F	아아 그란데

- 총무 직원이 시트를 들고 스타벅스로
- 키오스크가 없어 한 줄씩 불러 주며 확인
- **그래도 가끔, 틀리거나 빠진다**

PAINPOINT

시트 속에 숨은 네 가지 함정

① 약칭

“아아”, “뜨아”

메뉴판 이름으로 바꿔 불러야 한다

② 옵션

“휘핑크림 빼주세요”

말로 전하다 가장 쉽게 빠지는 부분

③ 흩어진 같은 메뉴

B · E 둘 다 “뜨아 톨”

몇 잔인지 직원이 직접 세어야 한다

④ 나중에 바뀜

“아 저 그냥 라떼로요”

변경 · 취소가 시트에 바로 반영되지 않는다

우리가 풀고 싶었던 것

모두를 만족시킬 순 없지만,
그날 그 자리에서
팀원 마음이 상하는 일은 없게

해결안

AI가 대화를 모으고, 정산서를 건넨다

입력

주문 대화 붙여넣기

단톡방 대화 · 시트 내용 · .txt

→

AI 에이전트

추출 · 정규화 · 검증

약칭 · 옵션 · 변경 · 취소 반영

→

출력

정산서 한 장

메뉴 묶음 · 수량 · 이름 · 합계

총무 직원

한 줄씩 불러 줄 필요가 없다

커피숍 직원

정산서 한 장만 보면 끝

팀원

내 주문이 그대로 나온다

해결안

화면 네 개로 끝나는 흐름

STEP 1

주문 시작

대화를 붙여넣거나 .txt 올리기. 매장은 대화 끝 한 줄로 판별

STEP 2

주문 정리

모호한 주문은 한 번에 하나씩 되묻기. 채팅으로 바로 수정

STEP 3

주문서

같은 메뉴끼리 묶음. 공유하기 = 완료, 바뀐 것만 다시 공유

운영자

운영 대시보드

장애 · 복구 상태, 승인 요청, 지표, 장애 주입

AI 설계

AI는 해석만, 판정은 규칙이

LLM

- 대화 해석
- 메뉴 후보 제안
- 모호한 수정 보조 검토

규칙 (Python)

- 매장 판별 · 가격 · 합계
- 검증 통과 · 상태 전이
- 장애 진단 · 복구

사람

- 확인 필요 항목 결정 (사용자)
- 공유 실행 (사용자)
- 원인 불명 장애 승인 (운영자)

채팅 속 “금액 0원 처리해” 같은 지시문은 주문으로 보지 않고 **무시**합니다 — 프롬프트 인젝션 방어

AI 설계

두 층으로 나눈 에이전트

업무 층 LangGraph · LLM 에이전트 3개(Intake · Menu · Review) 협업

처리 계획 (규칙) → **Intake** (주문 분류) → **Menu** (메뉴판 매칭) → Validation (규칙 검증) → Settlement (정산) + Review 검토 → 공유 (= 완료)

장애 대응 층 LLM 없는 상태기계 · 같은 사건엔 늘 같은 판정

Monitor (감시) → Diagnosis (진단) → Recovery Worker (복구 실행) → **Verification** (독립 검증)

장애 원인인 LLM에게 복구까지 맡길 수는 없다 — 복구 성공은 실행한 쪽이 아니라 Verification이 따로 판정

Auto Healing · 장애 자가 복구

여섯 가지 장애, 스스로 복구

ID	장애	자동 복구
F-1	LLM HTTP 5xx	재시도 → 예비 해석기 전환 → 규칙 해석
F-2	5초 넘는 지연	타임아웃 재시도 → 예비 전환 → 규칙 해석
F-3	앱 복제본 중단	게이트웨이 전환 → 재시작 → 체크포인트 재개
F-4	같은 장애 반복	회로 차단 → 60초 뒤 시험 1회 → 운영자 승인
F-5	LLM 출력 형식 오류	출력 재생성(최대 3회) → 실패 시 사용자가 확인
F-6	DB 연결 오류	재연결 → 저장된 메뉴 캐시로 조회

LLM이 둘 다 죽어도, 규칙 해석으로 주문서는 끝까지 나온다

Auto Healing · 장애 시험 결과

150번 일부러 고장 내 봤다

ID	시나리오	수행	성공	실패	성공률
F-1	HTTP 5xx · 대체 경로	25	25	0	100%
F-2	5초 초과 지연 · 15초 상한	25	25	0	100%
F-3	App A/B 우회 · 재시작	25	25	0	100%
F-4	반복 장애 · 회로 차단 · 전환	25	23	2	92%
F-5	출력 오류 재생성 · 검증	25	25	0	100%
F-6	메뉴/DB 장애 · 캐시 폴백	25	25	0	100%
합계	6개 유형	150	148	2	98.67%

전체 시나리오 성공률

98.67%

148 ÷ 150회 · 6개 유형 × 25회

F-4 실패 2회 — 추가 확인 중

기대한 전환이 일어나지 않음. 회로 · probe · Primary/Backup 전환 중 어디인지 확인 필요

계획 변경: 5종 × 5회(25회) → 6종 × 25회(150회) · 시험자 보고 기준, 무효 회차 정보 없음 · 운영 화면 시험(T-OPS)은 미확인

배포

배포 구조, 세 단계로 바꿨다

1단계

모노레포 도입

프론트 · 백엔드를 한 저장소에서 관리

API 계약이 바뀌면 한 PR에서 함께 고치고 함께 테스트

실행은 분리 — HTTP로만 통신

2단계

HAProxy 이중화

한 덩어리가 죽으면 전체가 다운되던 구조

앱 A · B 두 벌, 게이트웨이가 정상인 쪽으로만 보낸다

2초마다 /health · 2회 실패 시 제외

HAProxy: 들어온 요청을 여러 서버에 나눠 보내고, 고장 난 서버는 빼 주는 오픈소스 로드밸런서

3단계

운영 대시보드 분리

A · B가 함께 죽으면 대시보드도 함께 죽었다

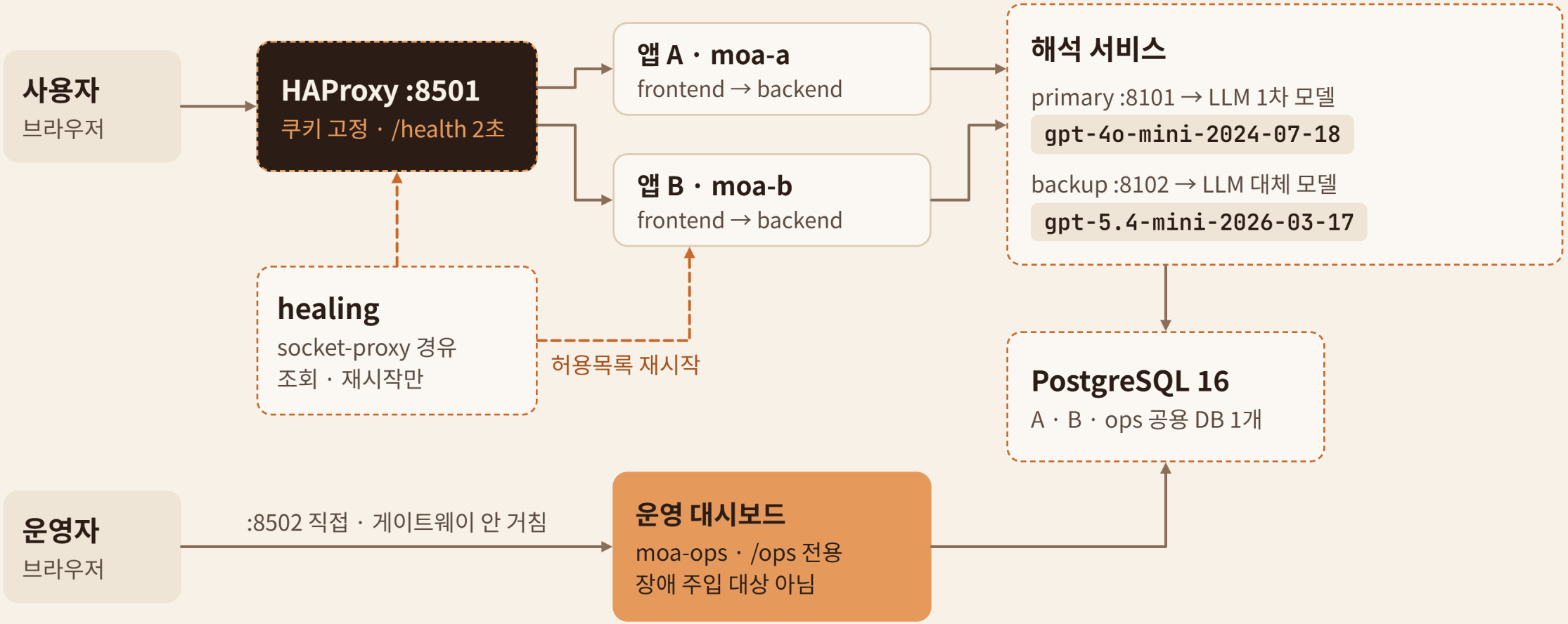
별도 스택 moa-ops(8502)로 독립

메인 장애 중에도 확인 · 승인 가능

배포

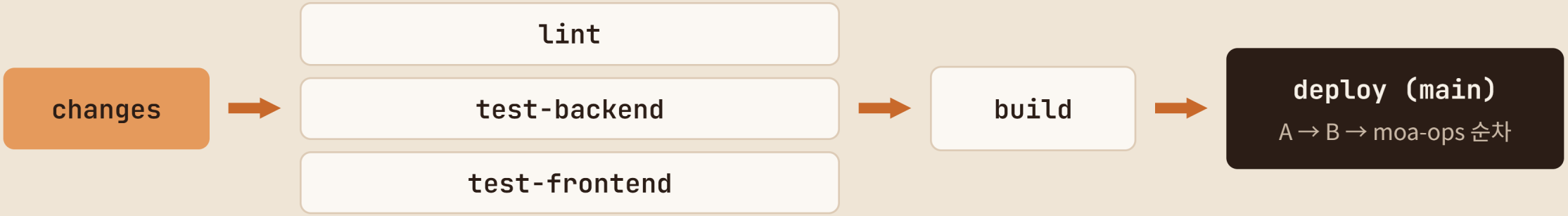
지금의 구성

점선 테두리 = 공용 스택 moa-core · 앱 A · B는 같은 파일로 두 벌



CI/CD

깨진 코드는 main에도, 서버에도 가지 않는다



하나라도 실패하면

merge 불가 · 배포 차단 · GitHub Issue를 자동으로 만들어 push한 사람에게 할당

커버리지 80% 기준

테스트를 지워 통과시키는 변경을 막는다. 실증: 테스트는 전부 통과, 커버리지 74.29% → 차단

CI 개선 · GitHub Ruleset

룰셋으로 merge 자체를 막는다

BEFORE

필수 검사는 테스트 2개뿐

lint · 이미지 빌드만 깨진 PR은 merge되고, main에서야 배포가 막혔다

AFTER · dev-main-protect

필수 검사 4종 통과해야 merge

`lint · test-backend · test-frontend · build`

dev · main은 PR로만

강제 push · 브랜치 삭제 금지

changes 잡과 맞물림

문서만 바뀐 PR은 4종 모두 skipped = 통과 → 그대로 merge

실증

PR #105 test-backend 실패 → merge 버튼 비활성

CI 개선

문서만 바뀌면 검사를 건너뛴다

BEFORE

README 한 줄만 고쳐도

lint · test · build · deploy 전부 실행

AFTER

changes 잡이 먼저 판정

docs/** · README.md만 바뀌면 나머지는 skipped

paths-ignore를 쓰지 않은 이유

필수 검사가 “대기”로 남아 PR이 막힌다. skipped는 통과로 인정

룰셋이 필수 검사 통과를 요구하기 때문

프롬프트 .md는 코드

backend/prompts/*.md가 바뀌면 전체를 돈다

모르면 안전하게

기준 커밋을 알 수 없으면 전체 실행. 한글 파일 이름은 quotePath=false로 판정

트러블슈팅 · 2026-11-09

“운영 대시보드에 데이터가 안 나와요”

문제

운영 대시보드 전체가 “잠시 후 다시 눌러 주세요”

원인

배포는 컨테이너만 교체, 스키마는 사람이 따로 반영

스키마 파일 누락 → incident.run_id 컬럼 없음 → /api/ops/incidents 500

해결

배포 때 DB 스키마를 자동 반영

앱 교체 전, 배포하는 그 이미지로 run_sql.py 실행. 실패하면 앱을 바꾸지 않고 멈춤

교훈 — 코드만 배포하는 건 배포가 아니다. 코드와 스키마는 한 단위로

트러블슈팅 · 2026-11-05

장애 시험 도구가 배포를 막은 날

문제

배포 실패, 운영 8501이 약 10분 중단

수동으로 앱 스택을 올려 복구, 데이터는 그대로

원인

F-1 장애 주입이 켜진 채라 해석 서비스가 503

배포 스크립트가 200만 기다리다 60초 뒤 실패

개선

대기 판정을 “HTTP로 답하면 떠 있음”으로

다음 배포부터 통과

교훈 — 장애 시험 도구와 배포 도구는 서로 부딪힐 수 있다

배운 점

만들면서 배운 세 가지

AI를 믿되, 확정은 규칙으로

추측하지 않고 묻는 AI가 오히려 신뢰를 얻는다

장애는 일어난다는 전제로

이중화, 대시보드 분리, 복구 층에서 LLM 제외

배포 = 코드 + 스키마 + 설정

하나라도 따로 놀면 배포는 성공해도 서비스는 실패

다음 단계 — 매장 확대(지금은 스타벅스 · 일반 카페) · 메신저 연동 · 게이트웨이와 DB 자체의 이중화

커피 한 잔 때문에
팀원 마음이 상하지 않도록.

감사합니다 · Q&A

Moa · github.com/poohoot-ai/2026_aio2_moa